# Crownfield Eye Hospital - Database Entity Relationship Diagram (ERD)

logical database design to show entities, attributes, keys, relationships, cardinality and Modality (01_schema)

**Legend**
PK = Primary key (unique row identifier)
FK = Foreign key (references another table)
UQ = Unique constraint (no duplicate value)
Cardinality (Crow's Foot Notation)
| = Exactly one (mandatory)
O| = Zero or one (optional)
< = Many
O< = Zero or many
|< = One or many
Relationship labels read parent → child. Child rows contain the foreign key.

Design Notes
• Appointment is the central transactional entity.
• Prescription_Item is a junction table resolving a many-to-many relationship.
• Follow_Up uses a UNIQUE FK to enforce one follow-up per appointment.
• Lookup tables reduce redundancy and support referential integrity.
• Schema is normalised to 3NF (UNF → 1NF → 2NF → 3NF).

KEY BUSINESS RULES
• Each patient may attend many appointments.
• Each appointment belongs to one patient.
• Each appointment is assigned to one clinician.
• Each appointment occurs at one clinic.
• One appointment may produce many diagnoses.
• One appointment may produce many prescriptions.
• One appointment may have at most one follow-up.

**patient**

| | | |
|---|---|---|
| PK | patient_id | VARCHAR(10) |
| | first_name | VARCHAR(50) |
| | last_name | VARCHAR(50) |
| | postcode | VARCHAR(10) |
| | date_of_birth | DATE |
| | phone | VARCHAR(20) |
| UQ | email | VARCHAR(100) |
| | created_at | TIMESTAMP |

**clinic**

| | | |
|---|---|---|
| PK | clinic_id INT | |
| | - clinic_name: VARCHAR(80) [UQ] | |
| | - address: VARCHAR(150) | |

A patient can attend many appointments. Each appointment belongs to one patient.

A clinic can host many appointments. Each appointment occurs in one clinic.

**specialty**

| | | |
|---|---|---|
| PK | specialty_id INT | |
| UQ | specialty_name | VARCHAR(60) |

A specialty can have many clinicians. Each clinician has one primary specialty.

**appointment**

| | | |
|---|---|---|
| PK | - appointment_id: INT [PK] | |
| FK | patient_id | VARCHAR(10) |
| | clinician_id | INT |
| FK | clinic_id | INT |
| | appointment_date | DATE |
| | appointment_time | TIME |
| | notes | VARCHAR(255) |
| | status | VARCHAR(20) |

An appointment may result in zero or many diagnoses.

**diagnosis**

| | | |
|---|---|---|
| PK | diagnosis_id | INT |
| FK | appointment_id | INT |
| | diagnosis_name | VARCHAR(100) |
| | diagnosis_notes | VARCHAR(255) |

**clinician**

| | | |
|---|---|---|
| PK | clinician_id INT | |
| | first_name | VARCHAR(50) |
| | last_name | VARCHAR(50) |
| FK | specialty_id | INT |
| UQ | email | VARCHAR(100) |

A clinician can conduct many appointments. Each appointment has one clinician.

An appointment may generate zero or one follow-up plan.(Enforced by UNIQUE appointment_id)

**follow_up**

| | | |
|---|---|---|
| PK | follow_up_id | INT |
| FK /UQ | appointment_id | INT |
| | due_date | DATE |
| | priority | VARCHAR(20) |
| | status | VARCHAR(20) |

An appointment may generate zero or many prescriptions.

OPTIONAL RELATIONSHIPS
Appointments may exist without diagnoses, prescriptions, or follow-ups.

**prescription**

| | | |
|---|---|---|
| PK | prescription_id | INT |
| FK | appointment_id | INT |
| | prescription_date | DATE |

A prescription contains one or more medicine items.

**prescription_item**

| | | |
|---|---|---|
| PK | prescription_item_id | INT |
| FK | prescription_id | INT |
| FK | medicine_id | INT |
| | dose | VARCHAR(50) |
| | route | VARCHAR(30) |
| | frequency | VARCHAR(50) |
| | duration_days | INT |

A medicine may appear in many prescription items.

**medicine**

| | | |
|---|---|---|
| PK | medicine_id | INT |
| UQ | medicine_name | VARCHAR(100) |

NORMALISATION SUMMARY
UNF
Multiple medicines stored in one field.
1NF
Atomic values only.
2NF
Prescription items separated.
3NF
Lookup tables remove transitive dependencies.

NORMALISATION NOTE
Prescription_Item is a junction table.
A prescription may contain many medicines.
A medicine may appear in many prescriptions.
This many-to-many relationship is resolved through Prescription_Item, which stores medicine-specific details including dose, route, frequency and duration.

How the relationships support the 6 queries (joins)

**Q1 • GROUP BY + HAVING**
Tables: clinician, appointment
**Join:** clinician INNER JOIN appointment
**Purpose:** Count appointments per clinician and use HAVING to filter grouped results e.g. clinicians with more than a chosen number of appointments.

**Q2 • 3-table INNER JOIN**
Tables: appointment, patient, clinician, clinic
**Joins:** appointment → patient, clinician and clinic
**Purpose:** Produce an appointment report showing the patient, treating clinician and clinic together.

**Q3 • LEFT JOIN**
Tables: appointment, patient, diagnosis
**Joins:** patient INNER JOIN appointment; LEFT JOIN diagnosis
**Purpose:** Include appointments even when no diagnosis has been recorded (diagnosis columns return NULL).

**Q4 • Subquery**
Tables: clinician, appointment
**Technique:** Compare each clinician's appointment count with a scalar/aggregate subquery (e.g. above the average).
**Purpose:** Identify clinicians with higher-than-average appointment activity.

**Q5 • Stored procedure**
Tables: appointment, patient, clinician, clinic
**Technique:** IN parameter for patient_id; joins plus WHERE and ORDER BY.
**Purpose:** Reusable appointment-history lookup for one patient.

**Q6 • Trigger**
Tables: follow_up, appointment
**Event:** BEFORE INSERT on follow_up.
**Logic:** Compare NEW.due_date with the related appointment_date; use SIGNAL SQLSTATE to reject an invalid date.